Guía de exposición y defensa técnica

Proyecto Integrador: Pipeline ETL Robusto con Zona de Cuarentena Ingeniería de Datos (ATD-2602) · TecNM Campus Colima · Grupo 1 (LCS2), Retail omnicanal Prof. Armando Román Gallardo Heredia · Exposición: 29 de octubre de 2026, 14:00 h Equipo (orden alfabético): Angel Salvador Alvarez · Gael Alexandro Cortez Sarmienta · Sebastian Grageda Sandoval

---

0. Las cifras que hay que saber de memoria

Dato	Valor
Registros crudos	1,220 (620 tienda física + 600 online)
Tras limpieza e integración (fases 1 a 4)	1,189
Validados → `transacciones_limpias`	**1,049 (88.23 %)**
Desviados → `cuarentena_errores`	**140 (11.77 %)**
Monto facturable validado	**$4,074,888.52 MXN** (77.7 % tienda física, 22.3 % online)
Pruebas de estrés del esquema Pydantic	25 casos, **0 excepciones no controladas**
Suite de QA con verdad conocida	9 escenarios, **200 verificaciones, 0 fallos**
Pruebas de tortura del motor universal	21 archivos raros o rotos, **0 excepciones no controladas**
Volumen máximo probado	**1,000,000 de filas en 40 s** (en memoria) · archivos de **más de 5 GB por bloques**
Suite de extensiones	`config.yaml` , bloques = memoria byte a byte, panel en vivo: **59 verificaciones, 0 fallos**

Causas de la cuarentena: 57 llaves huérfanas · 37 correos con sintaxis inválida · 25 totales que no cuadran · 11 correos con `@@` o `..` · 8 bots (total inflado ×80) · 2 duplicados de negocio.

> Si una cifra no la recuerdas, se lee en vivo en la pestaña **Resumen** del panel. Nunca inventes un número: di "eso lo vemos en el panel" y ábrelo.

---

1. De qué trata el proyecto, de inicio a fin

1.1 Qué pidió el profesor

Diseñar, implementar y defender un **pipeline ETL** en Python que:

1. ingiera datos crudos e inconsistentes de **fuentes heterogéneas** (CSV de tienda física + JSON de tienda web);
2. **diagnostique** su calidad con métricas objetivas;
3. aplique **preprocesamiento estadístico** (faltantes, outliers, escalamiento, discretización);
4. **valide** las reglas de negocio con **Pydantic**;

5. **persista** lo válido en una base relacional (SQLite) con transacciones seguras;
6. **aísle** lo inválido en una **tabla de cuarentena** con su motivo, **sin detener el lote**.

El principio de evaluación es la **robustez**: no basta con que funcione en el caso ideal; tiene que soportar registros corruptos sin colapsar ("no crash") y dejar trazabilidad de cada anomalía.

1.2 Qué se necesitó

Elemento	Qué es
Datos	`catalogo_productos.csv` (maestro, 50 productos), `ventas_tienda_fisica.csv` (620), `ventas_online.json` (600)
Retos del dataset	POS duplicados, fechas mezcladas (DD/MM, DD-MM, ISO, epoch), totales nulos, precios en USD, correos malformados, bots con compras gigantes, productos inexistentes
Lenguaje y librerías	Python 3.10+, pandas, numpy, scipy (pruebas estadísticas), pydantic v2 (validación), sqlite3 (estándar)
Equipo y roles	Ingeniero 1: calidad, nulos y outliers · Ingeniero 2: transformación e integración · Ingeniero 3: validación, resiliencia y base de datos
Entregables	Repositorio Git, `REGLAS_DE_NEGOCIO.md` justificado, base `pipeline.db`, reportes y defensa oral

1.3 Qué construimos

```
datasets/ → Fase 0 contrato de esquema → Fases 1-2 diagnóstico e imputación
          → Fases 3-4 outliers e integración → Fases 5-6 escalamiento y validación
          → Fase 7 carga a SQLite (transacciones_limpias + cuarentena_errores)
```

Además, tres piezas que van más allá de lo pedido:

- **Motor universal** (`motor/pipeline_universal.py`): las mismas 7 fases sobre **cualquier dataset**, sin conocer sus columnas. Es lo que permite que el profesor suba el suyo.
- **Panel web** (`app/`): ejecuta el pipeline, muestra cada fase y recibe archivos por arrastrar y soltar. Sin dependencias externas.
- **Suites de prueba** (`qa/`): 200 verificaciones con datos de verdad conocida, 21 pruebas de tortura y una carga de un millón de filas.

---

2. Explicación simple (para abrir la exposición, 45 segundos)

Imagina **dos cuadernos de ventas**. En uno anota a mano la cajera de la tienda; el otro lo llena solo la página web. Venden lo mismo, pero escriben distinto: uno pone la fecha "13-09-2026" y el otro un número larguísimo; uno cobra en pesos y el otro a veces en dólares; a veces una venta quedó apuntada dos veces y a veces aparece un producto que ni existe.

Si juntas los dos cuadernos tal cual para sacar el total, el reporte sale mal. Lo que hicimos es **una banda transportadora con siete filtros**: traduce los nombres, revisa qué falta, rellena con criterio, baja el volumen a las ventas exageradas sin tirarlas, pone todas las fechas y monedas igual, clasifica cada venta en barata, media o cara, y al final un **guardia** revisa venta por venta.

La que está bien, pasa. La que está mal **no se tira**: va a un cuarto aparte, la **cuarentena**, con una nota que dice exactamente qué tenía mal. Un sistema malo se cae con una hoja mal escrita; el nuestro la aparta, la anota y **sigue trabajando**.

---

3. Explicación técnica, fase por fase

Fase	Qué hace	Decisión que hay que saber defender
0. Contrato de esquema	Traduce cualquier nombre de columna al canónico: exacto, alias (español/inglés/ERP) o coincidencia difusa	Si falta una columna obligatoria, se detiene **sin escribir nada**: mejor no producir salida que producir una salida silenciosamente incorrecta
1. Diagnóstico	Completitud, unicidad y tipos inferidos	Cuenta los **vacíos ocultos** (`"NA"` , `"N/A"` , `""`), que `isna()` no ve
2. Faltantes	Clasifica MCAR/MAR con chi² y binomial con Bonferroni; imputa	**Recalcular antes que imputar** (`monto = cantidad × precio` es exacto); mediana porque los importes están sesgados; etiqueta `ANONIMO` y no la moda, porque la moda inventa un dato
3. Outliers	IQR con factor 1.5 y **capping**	No se borra ninguna fila por ser extrema; solo se eliminan **imposibles** (cantidad ≤ 0). USD→MXN **antes** del IQR para no mezclar escalas
4. Integración	Fechas a ISO con formatos explícitos por fuente; epoch en UTC → hora de México; USD × 19.50; merge con `indicator=True` contra el catálogo	49 de 208 epoch caerían en otro día si se tomaran en UTC. Los huérfanos se **marcan**, no se borran
5. Transformación	Min-Max a [0, 1]; discretización BAJO/MEDIO/ALTO por terciles	Min-Max **después** del capping: con el máximo original ($2.1 M) el 99 % quedaba bajo 0.01. Terciles y no cortes fijos porque el catálogo mezcla precios muy distintos
6. Validación	`BaseModel` con `@field_validator` y `@model_validator` ; `try/except ValidationError` por registro	Cada regla cruzada nace de un hallazgo forense (sección 4)
7. Carga	SQLite con `PRIMARY KEY` , `CHECK` , `NOT NULL` ; `BEGIN` → `DELETE` → `executemany` → `commit` , `rollback` ante error	El rollback **se prueba contra la base real**: se inserta `cantidad = -99` , salta el CHECK y el conteo no cambia

Reglas cruzadas de la Fase 6

Regla	Qué exige
R1	La llave de producto no puede ser huérfana
R2	El precio no puede ser 500.00 exacto (valor de relleno del POS)
R3a	Si la Fase 2 marcó el total como inconsistente, va a cuarentena aunque el capping lo haya "arreglado" (bots ×80)
R3b	`monto = cantidad × precio` (± $0.01) en filas no recortadas
R4	Estatus CANCELADO o DEVUELTO ⇒ `monto_facturable_mxn = 0`
R5	Toda venta online necesita correo válido

La historia de R3a vale oro en la defensa: la Fase 3 recortaba el total inflado ×80 de los bots y lo marcaba como outlier; como la regla aritmética se salta las filas recortadas, **8 de los 12 bots entraban a la tabla limpia con un monto falso**. Se detectó, se corrigió con la bandera de la Fase 2 que sobrevive al capping, y ahora hay **0 bots** en la tabla limpia. Es la prueba de que el pipeline se audita a sí mismo.

---

4. El diferenciador: datos que parecen correctos pero no lo son

Cód.	Hallazgo	Casos	Por qué engaña
E-01	Total ≠ cantidad × precio	12	Los tres campos son números válidos
E-02	Bots: total inflado **exactamente ×80**	12	Parecen ventas grandes legítimas
E-03	Precio **500.00 exacto**	29	Importe plausible; lo delata repetirse 29 veces
E-04	Llaves huérfanas con formato perfecto	62	Cumplen `PROD-NNN` y pasan cualquier regex
E-05	`usuario@@gmail..com`	11	Tiene arroba, dominio y `.com`
E-06	Duplicados de negocio con folio distinto	40	`drop_duplicates()` no los ve
E-09	Filas que **nuestro** capping dejó incoherentes	40	La incoherencia la causamos nosotros
E-10	Cancelados que conservan importe	119	Cada campo es válido; la contradicción está en leerlos juntos

E-10 no se manda a cuarentena: son el 10 % del lote y un pedido cancelado es un hecho real. Se resuelve con `monto_facturable_mxn = 0`: **se conserva el dato y se corrige cuánto factura**.

---

5. La aplicación y la demostración en vivo

5.1 Cómo se abre

Opción	Cómo	Cuándo usarla
Panel en vivo	Doble clic en `INICIAR_PANEL.bat` (o `python app/servidor.py`)	Exposición normal. Abre solo el navegador
Respaldo offline	Doble clic en `dashboard_offline.html`	Si falla Python, el puerto o la laptop. No necesita servidor ni internet

El panel **no usa ninguna librería externa**: servidor con la librería estándar de Python y gráficas SVG escritas a mano. Sin internet funciona igual.

5.2 Pestañas

Prueba tu dataset (subida de archivos) · Resumen · Las 7 fases · **Antes / después** · Datos engañosos · Cuarentena (filtrable) · Negocio · Pruebas de estrés · QA a escala · Ejecutar pipeline.

Antes / después (nueva): columna por columna, el dato crudo contra `transacciones_limpias`. Qué mostrar: `monto_mxn` (máximo 2,109,055 → 16,510.02 por el capping; 11 valores ≤ 0 → 0; el diagrama de caja en escala logarítmica), `fecha` (5 formatos → solo AAAA-MM-DD) y `correo` (28 inválidos y 20 vacíos → 0: son los 48 correos de la

cuarentena). Tiene filtro por canal. Después de subir un dataset, el botón «Ver antes / después por columna» hace lo mismo con el archivo del profesor.

5.3 Qué decirle al profesor para la demostración

> "Profe, si gusta, suba usted cualquier dataset: limpio o sucio, CSV, Excel, JSON o un ZIP con varios. El sistema no necesita saber cómo se llaman sus columnas."

Qué pasa cuando sube un archivo:

1. **Lectura:** detecta formato, codificación (UTF-8, Windows-1252, Latin-1), separador (`,` `;` tabulador `|`), coma o punto decimal, todas las hojas de un Excel y JSON anidado.
2. **Fase 0:** nombres de columna limpios y únicos, filas y columnas vacías fuera, vacíos ocultos a nulo.
3. **Inferencia de tipos:** numérico, fecha, correo, booleano, identificador, categórico o texto.
4. **Fases 1 a 7** completas, con la misma lógica del proyecto: MCAR/MAR, recálculo, mediana, IQR, fechas ISO, USD→MXN, Min-Max, terciles, Pydantic y SQLite.
5. **Si sube varios archivos**, detecta solo cuál es el catálogo maestro (una columna con llaves únicas que otra tabla usa) y marca los huérfanos.
6. **Resultado:** KPIs, una tarjeta por fuente con cada fase, causas de cuarentena y descargas (CSV limpio, CSV de cuarentena, base SQLite, reporte JSON).

Prueba hecha con los tres archivos oficiales **sin decirle nada al motor sobre sus columnas**: encontró solo las relaciones `product_id → id_producto` e `id_producto → id_producto`, los 62 huérfanos, los 12 bots, los correos inválidos, los imposibles y los duplicados.

5.4 Qué lo hace robusto ("modo paranoico")

Riesgo	Protección
Archivo enorme	Escritura a disco por bloques; validación Pydantic por lotes de 50,000 con `TypeAdapter` (núcleo en Rust); **1 M de filas en 40 s**
Archivo vacío, binario o JSON roto	Se detiene de forma controlada con mensaje; no escribe nada
Líneas con columnas de más	Se omiten y se reportan
Columnas duplicadas o sin nombre	Se renombran (`monto` , `monto_1` , `columna_3`)
Coma decimal (`1.234,56`) o `$1,234.50` o `(500)`	Se decide por columna, no por valor
Fechas mezcladas, epoch, fechas imposibles (31/02)	Formatos explícitos; lo no convertible se marca y va a cuarentena con motivo
Infinitos o `1e308`	Pydantic con `allow_inf_nan=False`
Una fase falla con un archivo raro	Se registra en la bitácora y se continúa con el resto
`NaN` dentro de las respuestas	Se sanea antes de enviarlo al navegador
Subidas con nombres maliciosos	Nombre saneado, carpeta por corrida, rutas validadas

6. Pruebas que respaldan la robustez

Suite	Qué verifica	Resultado
Estrés del esquema (`fase5_fase6_pipeline.py`)	25 registros corruptos inyectados al modelo Pydantic	0 excepciones; solo el control válido pasa
QA con verdad conocida (`qa/suite_qa.py`)	9 escenarios: normal, 3× sucio, limpio, columnas extra, columnas faltantes, esquema en inglés, esquema ERP, 25 k y 100 k	**200 verificaciones, 0 fallos**
Tortura del motor (`qa/tortura_universal.py`)	21 archivos: Excel multihoja, CP-1252 con `;`, TSV con `$`, JSON anidado, JSON Lines, Parquet, ZIP, binario, vacío, JSON roto, celdas de 2 MB, infinitos, 1 M de filas	**0 excepciones no controladas**
Rollback	Insertar una fila que viola `CHECK`	Tabla intacta
Extensiones (`qa/suite_extensiones.py`)	`config.yaml` (lectura, errores de dedo, cambio de umbral de punta a punta), modo por bloques comparado byte a byte contra el modo en memoria, falla a media carga sin salidas parciales y panel en vivo	**59 verificaciones, 0 fallos**

Los **10 invariantes** de integridad que se comprueban en cada escenario: ninguna fila se pierde · cero huérfanos en limpias · cero correos inválidos · aritmética coherente · fechas ISO · cero USD sin convertir · normalizado en [0, 1] · llave única · toda cuarentena con motivo · cero montos no positivos. Más idempotencia y recarga a BD sin duplicar.

Lo que las pruebas **encontraron y se corrigió**: el `KeyError` con otros nombres de columna (→ Fase 0), el `TypeError` al imputar texto en una columna 100 % vacía, los bots que se colaban por el capping (→ R3a), la coma decimal mal leída y el precio imputado que no cuadraba con el total (→ recalcular antes de imputar).

---

7. Guion de 5 minutos (`PRESENTACION_EXPOSICION_5MIN.pptx`)

Nueve diapositivas; el guion completo de cada una está en las **notas del orador** del PowerPoint. Los nombres de la portada van en orden alfabético por primer apellido; el reparto de diapositivas es una sugerencia: ajústenlo al rol que cada quien tuvo.

Tiempo	Quién	Diapositiva	Idea que tiene que quedar
0:00 – 0:10	Integrante 1	1 · Portada	Pipeline ETL con cuarentena: ningún registro corrupto detiene el lote
0:10 – 0:40	Integrante 1	2 · El problema	3 fuentes, 1,220 registros y siete tipos de suciedad; 0 excepciones
0:40 – 1:10	Integrante 1	3 · Arquitectura	1,220 → 1,189 → 1,049 limpias + 140 en cuarentena; 1,049 + 140 = 1,189
1:10 – 1:45	Integrante 1	4 · Fases 1–3	Recalcular antes de imputar, mediana (16,991 contra 3,622), capping 2.1 M → 16,510
1:45 – 2:30	Integrante 2	5 · Fases 4–5	Hora de México (49 de 208), USD a 19.50 antes del IQR, 27 + 33 huérfanos, terciles
2:30 – 3:05	Integrante 3	6 · Fase 6	Reglas R1–R5 nacidas del forense; R3a: 8 de 12 bots se colaban; 140 con motivo
3:05 – 3:35	Integrante 3	7 · Fase 7 y robustez	Commit/rollback probado con −99; 25 de estrés, 200 verificaciones, 21 torturas: 0 fallas
3:35 – 4:10	Quien hace la demo	8 · Más allá de lo pedido	Motor universal, más de 5 GB por bloques (5.29 GB probado), config.yaml, Antes / después
4:10 – 5:00	Quien hace la demo	9 · Resultado y demo	$4,074,888.52; tres conclusiones; demo: Resumen → Antes / después → Prueba tu dataset

Frase de cierre: *"Una cuarentena bien diseñada aísla lo que no se puede confiar, no todo lo que no encaja."*

Para la demo, si el profesor no trae archivo, usar `demo/profe_sucio.xlsx` (5,000 filas con 31 bots, 21 correos malos y 11 imposibles plantados: el motor encuentra exactamente esos).

---

8. Banco de preguntas

Nulos, imputación y perfilado (sesiones 21 a 23)

¿Por qué mediana y no media? Los importes están sesgados a la derecha. Con `[100, 200, 300, 10000]` la media da 2,650 y la mediana 250. El punto de ruptura de la mediana es 50 %; el de la media, 0 %.

¿Cómo distinguieron MCAR de MAR? Con chi-cuadrada entre la indicadora de ausencia y cada variable categórica observada, más binomial por nivel con Bonferroni. $p < 0.05 \Rightarrow$ MAR. Si no hay evidencia, se asume MCAR, y se dice así: MNAR no se puede probar con los datos observados.

¿Recalcular no es imputar? No. Imputar es estimar; `cantidad × precio` es exacto. La información sí está en el registro.

¿Por qué no la moda en categóricas? Porque inventa una pertenencia. `ANONIMO` dice la verdad: no se capturó.

¿Qué es un vacío oculto? Texto que significa "faltante" (`"N/A"`, `"null"`, `"-"`) y que `isna()` no detecta.

Outliers y ruido (sesiones 24 a 26)

¿Por qué capping y no borrar? Una venta de $27,000 puede ser real. El capping quita el peso del extremo sin perder la transacción. Solo se borran los imposibles.

¿Por qué IQR y no Z-score? El Z-score usa media y desviación, que están contaminadas por los mismos outliers. Los cuartiles son robustos.

¿Por qué 1.5? Convención de Tukey (~99.3 % de una normal). Está en `umbrales_iqr.csv` para recalcular con otro factor.

¿Y el suavizado SMA/EMA? Es para series continuas con ruido de medición. Cada fila aquí es una transacción independiente; promediar ventas destruiría el dato. Se aplicaría a la serie diaria agregada.

Normalización, pipelines y discretización (sesiones 27 a 29)

¿Min-Max no es sensible a outliers? Sí; por eso va después del capping. El orden de las fases es una decisión.

¿Y si llega un monto mayor al máximo? Queda fuera de [0, 1]. Los parámetros se guardan en `parametros_escalamiento.json`; en producción se recorta al rango o se reajusta programadamente.

¿Por qué terciles? Grupos de tamaño comparable (403/391/395). Para un reporte a dirección convendrían cortes de negocio: es una decisión de audiencia.

¿Cómo se relaciona con la sesión de pipelines? Cada fase es un paso independiente que recibe la salida del anterior y deja su resultado en disco, igual que un `Pipeline`: reproducible, auditable y reanudable.

Reducción e integración (sesiones 30 y 31)

¿Dónde está la reducción de datos? La discretización reduce la cardinalidad del importe a 3 niveles, y el panel agrega por día y por segmento.

¿Dónde está la integración heterogénea? Fase 0 (mapeo de esquemas) y Fase 4 (fechas, moneda y resolución de entidades contra el catálogo). El motor universal además detecta solo qué tabla es el maestro.

Validación y base de datos

¿Por qué Pydantic? Validadores declarativos que se leen como el contrato de negocio; `ValidationError` trae el campo exacto que falló (alimenta el motivo); separa campo de fila completa, que es donde viven los errores silenciosos.

¿Un registro con dos errores? La etiqueta es el primer error; `motivo_rechazo_cuarentena` guarda todos.

¿Qué se hace con la cuarentena? Es una bandeja de trabajo: corregir correos y reingresar, dar de alta productos en el catálogo, investigar los bots.

¿Por qué SQLite? No requiere servidor, viaja en el repositorio y usa SQL estándar. Migrar a PostgreSQL es cambiar la conexión.

¿Cómo garantizan la atomicidad? `BEGIN`, `commit` al final, `rollback` ante cualquier excepción, y la prueba con `cantidad = -99`.

Robustez y el motor universal

¿Qué pasa si subo un archivo con otras columnas? El motor infiere tipos y aplica las fases sin conocer los nombres. Probado con esquema en inglés, tipo ERP y archivos arbitrarios.

¿Y si subo algo que no es una tabla? Se detiene de forma controlada, lo dice y no escribe nada.

¿Cuánto aguanta? En memoria, un millón de filas en 40 s. Los archivos de más de 256 MB se procesan **por bloques** (`chunksize`) en tres pasadas: estructura, estadísticas globales y transformación. La memoria queda acotada aunque el archivo pese 5 GB o más, y el resultado es idéntico al del modo en memoria, verificado byte a byte.

¿Por bloques no cambia el IQR o los terciles? No. Los cuartiles, la mediana, el mínimo, el máximo y los cortes se calculan una vez con **todo** el archivo (pasada 2) y se aplican igual a cada bloque. Si cada bloque calculara sus propios cuartiles, el mismo monto se recortaría distinto según el bloque donde cayera.

¿Cómo saben que no corrompen datos? Generador con verdad conocida + 10 invariantes en cada escenario.

¿Qué harían distinto? Empezar por el contrato de esquema y por las pruebas.

¿Dónde están los umbrales? En `config.yaml`, versionado en git: tipo de cambio 19.50, IQR 1.5, terciles, alfa 0.05 y las reglas de la Fase 6. Cambiar un umbral es un commit con autor y motivo. El cargador rechaza errores de dedo (por ejemplo `factor_iqrr`) y valores absurdos (un factor IQR negativo); una fase con configuración inválida se detiene sin escribir nada.

¿Lo más difícil? Descubrir que nosotros mismos rompíamos datos: el capping dejó 40 filas incoherentes y dejaba pasar 8 bots. Un pipeline debe auditarse a sí mismo.

¿Por qué no hicieron el Grupo 2 (banca)? Nos tocó el Grupo 1, Retail. El código conserva la configuración del Grupo 2 y la omite si no están los archivos; el motor universal procesa los archivos bancarios igual.

Unidad 4 (sesión 40, recopilación especializada)

¿Cómo conecta con lo que sigue? Las fuentes especializadas (APIs de redes sociales, geoespaciales, clima o finanzas) llegan como JSON anidado y series con huecos. El motor ya aplana JSON anidado, convierte epoch y valida formatos: es la base para ingerirlas.

---

9. Checklist antes de exponer

- [ ] Doble clic en `INICIAR_PANEL.bat` y verificar que abre en la pestaña **Prueba tu dataset**.
- [ ] Tener `dashboard_offline.html` y `demo/profe_sucio.xlsx` a la mano.
- [ ] Si se va a usar Excel en la demo: `pip install -r requirements.txt` (instala `openpyxl`).
- [ ] Los tres integrantes repasan las cifras de la sección 0 y **todas** las fases, no solo la suya.
- [ ] Tener abierto `fase5_fase6_pipeline.py` por si piden ver el esquema Pydantic.

10. Si algo falla

Problema	Qué hacer
El panel no abre	`dashboard_offline.html` con doble clic
No hay internet	No importa: nada depende de la red
El profesor sube un Excel y falla	Falta `openpyxl`: "guarde como CSV" o `pip install openpyxl`
El archivo del profesor se detiene	Es la demostración de robustez: se detuvo con mensaje y sin escribir nada
No sabes una respuesta	"No lo medimos; lo que sí medimos es…"

11. Comandos

```
pip install -r requirements.txt
python app/servidor.py                                  # panel
```

```
python motor/pipeline_universal.py <archivo_o_carpeta>   # motor por consola
python qa/suite_qa.py --completo                         # 200 verificaciones
python qa/tortura_universal.py --grande                  # 21 torturas + 1 M de filas
python qa/suite_extensiones.py                           # config.yaml, bloques y panel
python configuracion.py                                  # umbrales vigentes
```